GoRent

Team 7777

Guillermo Mora

Tyler Rosa

Mim Dey

Thierno Sow

Muhammad Tufail

Rishika Datta

Introduction

GoRent is a peer-to-peer car rental platform that connects car owners with renters looking for affordable and flexible transportation. Owners can earn income from their vehicles while renters gain access to convenient transportation options.



Technology Stack



Postgresql Database

Relational data storage managing users, vehicles, and transactions using supabase



React/Next.js Frontend

Dynamic user interface with HTML, CSS, and JavaScript components

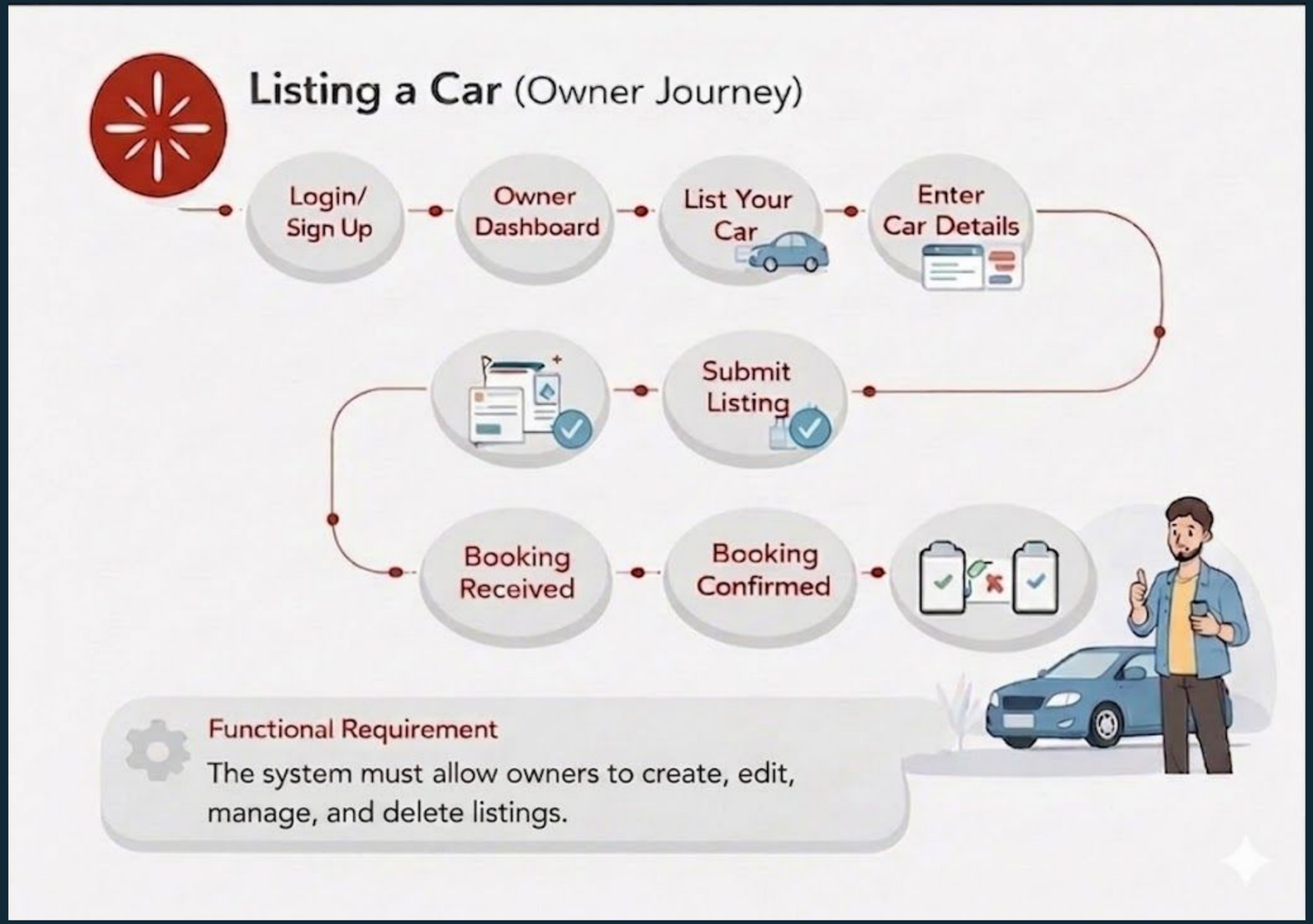


Python + Flask + Next.js

Backend API handling business logic and request processing

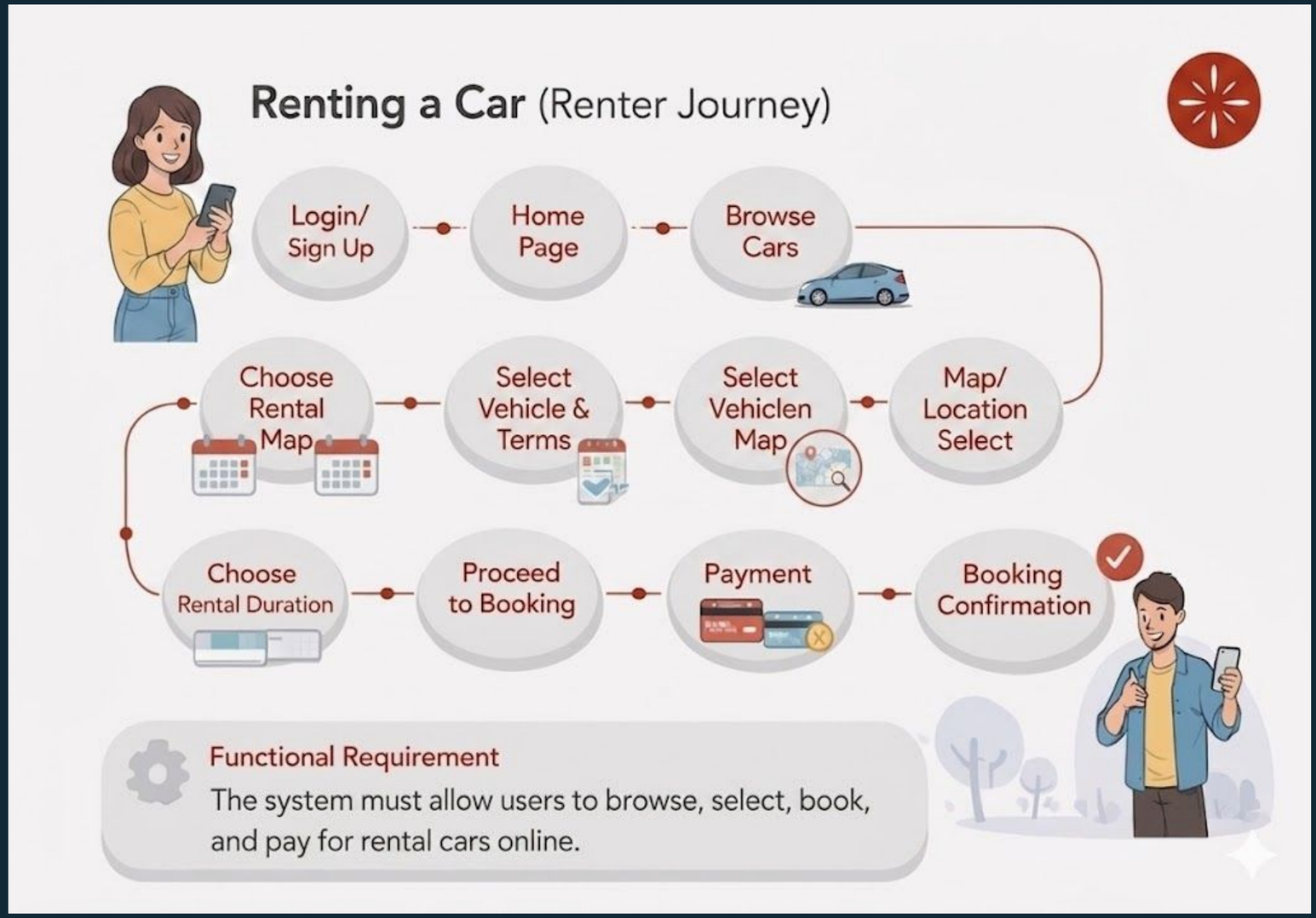
Functional Use Cases

Use Case 1: Vehicle Owner



Functional Use Cases

Use Case 2: Vehicle Renter



Stored SQL Procedures

Procedure 1: complete_booking_payment

```
-- Stored procedure 1: mark a booking as completed and insert a completed payment.
-- The payment trigger will automatically create the Revenue row.
CREATE OR REPLACE PROCEDURE complete_booking_payment(
    p_booking_id INTEGER,
    p_payment_method VARCHAR DEFAULT 'card',
    p_transaction_reference VARCHAR DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_total NUMERIC(10, 2);
BEGIN
    SELECT total_amount
    INTO v_total
    FROM "Booking"
    WHERE booking_id = p_booking_id;

    IF v_total IS NULL THEN
        RAISE EXCEPTION 'Booking % was not found or does not have a total_amount', p_booking_id;
    END IF;

    UPDATE "Booking"
    SET booking_status = 'completed'
    WHERE booking_id = p_booking_id;

    INSERT INTO "Payment" (
        booking_id,
        amount,
        payment_method,
        payment_status,
        transaction_reference
    )
    VALUES (
        p_booking_id,
        v_total,
        p_payment_method,
        'completed',
        p_transaction_reference
    );
END;
$$;
```

- Marks a booking as completed and inserts payment information into Payment table

Stored SQL Procedures

Procedure 2: cancel_booking_and_release_vehicle

```
150  -- Stored procedure 2: cancel a booking and release the related vehicle when no active bookings remain.
151  CREATE OR REPLACE PROCEDURE cancel_booking_and_release_vehicle(
152      p_booking_id INTEGER
153  )
154  LANGUAGE plpgsql
155  AS $$
156  BEGIN
157      UPDATE "Booking"
158      SET booking_status = 'canceled'
159      WHERE booking_id = p_booking_id;
160  END;
161  $$;
162
```

- Changes the status of an existing booking to “canceled”

Upon execution of both of these procedures, certain triggers activate and call functions to handle other updates to the database, shown in next slides.

SQL Functions

Function 1: Create Revenue For Payments

1. Declaration of Variables:

```
-- 1) Trigger function: automatically create/update a Revenue row when a payment is completed.
CREATE OR REPLACE FUNCTION create_revenue_for_completed_payment()
RETURNS TRIGGER AS $$
DECLARE
    v_booking_total NUMERIC(10, 2);
    v_booking_fee NUMERIC(10, 2) := 5.00;
    v_commission NUMERIC(10, 2);
    v_dropoff_fee NUMERIC(10, 2) := 0.00;
    v_platform_payout NUMERIC(10, 2);
    v_lister_payout NUMERIC(10, 2);
```

2. Procedure logic:

```
BEGIN
    IF LOWER(COALESCE(NEW.payment_status, '')) <> 'completed' THEN
        RETURN NEW;
    END IF;

    SELECT COALESCE(b.total_amount, NEW.amount)
    INTO v_booking_total
    FROM "Booking" b
    WHERE b.booking_id = NEW.booking_id;

    v_booking_total := COALESCE(v_booking_total, NEW.amount, 0);
    v_commission := ROUND((v_booking_total * 0.10)::NUMERIC, 2);
    v_platform_payout := ROUND((v_booking_fee + v_commission + v_dropoff_fee)::NUMERIC, 2);
    v_lister_payout := ROUND(GREATEST(v_booking_total - v_platform_payout, 0)::NUMERIC, 2);
```

Calculates total cost, commission, platform payout, and lister payout

3. Insert calculated values to Revenue Table:

```
    INSERT INTO "Revenue" (
        payment_id,
        booking_fee,
        commission,
        dropoff_fee,
        platform_payout,
        lister_payout,
        payout_date
    )
    VALUES (
        NEW.payment_id,
        v_booking_fee,
        v_commission,
        v_dropoff_fee,
        v_platform_payout,
        v_lister_payout,
        CURRENT_TIMESTAMP
    )
    ON CONFLICT DO NOTHING;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

4. Trigger for Procedure: On payment

```
CREATE TRIGGER trg_create_revenue_for_completed_payment
AFTER INSERT OR UPDATE OF payment_status, amount ON "Payment"
FOR EACH ROW
EXECUTE FUNCTION create_revenue_for_completed_payment();
```

SQL Functions

Function 2: Update Vehicle Status after a Booking Change

```
63  -- 2) Trigger function: keep Vehicle status in sync when bookings become active or canceled.
64  CREATE OR REPLACE FUNCTION sync_vehicle_status_after_booking_change()
65  RETURNS TRIGGER AS $$
66  DECLARE
67      v_vehicle_id INTEGER;
68  BEGIN
69      SELECT l.vehicle_id
70      INTO v_vehicle_id
71      FROM "Listing" l
72      WHERE l.listing_id = NEW.listing_id;
73
74      IF v_vehicle_id IS NULL THEN
75          RETURN NEW;
76      END IF;
77
78      IF LOWER(COALESCE(NEW.booking_status, '')) IN ('pending', 'confirmed', 'completed') THEN
79          UPDATE "Vehicle"
80          SET vehicle_status = 'unavailable'
81          WHERE vehicle_id = v_vehicle_id;
82      ELSIF LOWER(COALESCE(NEW.booking_status, '')) IN ('cancelled', 'canceled') THEN
83          IF NOT EXISTS (
84              SELECT 1
85              FROM "Booking" b
86              JOIN "Listing" l ON b.listing_id = l.listing_id
87              WHERE l.vehicle_id = v_vehicle_id
88                AND b.booking_id <> NEW.booking_id
89                AND LOWER(COALESCE(b.booking_status, '')) IN ('pending', 'confirmed', 'completed')
90          ) THEN
91              UPDATE "Vehicle"
92              SET vehicle_status = 'available'
93              WHERE vehicle_id = v_vehicle_id;
94          END IF;
95      END IF;
96
97      RETURN NEW;
98  END;
99  $$ LANGUAGE plpgsql;
```

Declaration of Variable

initialize variable

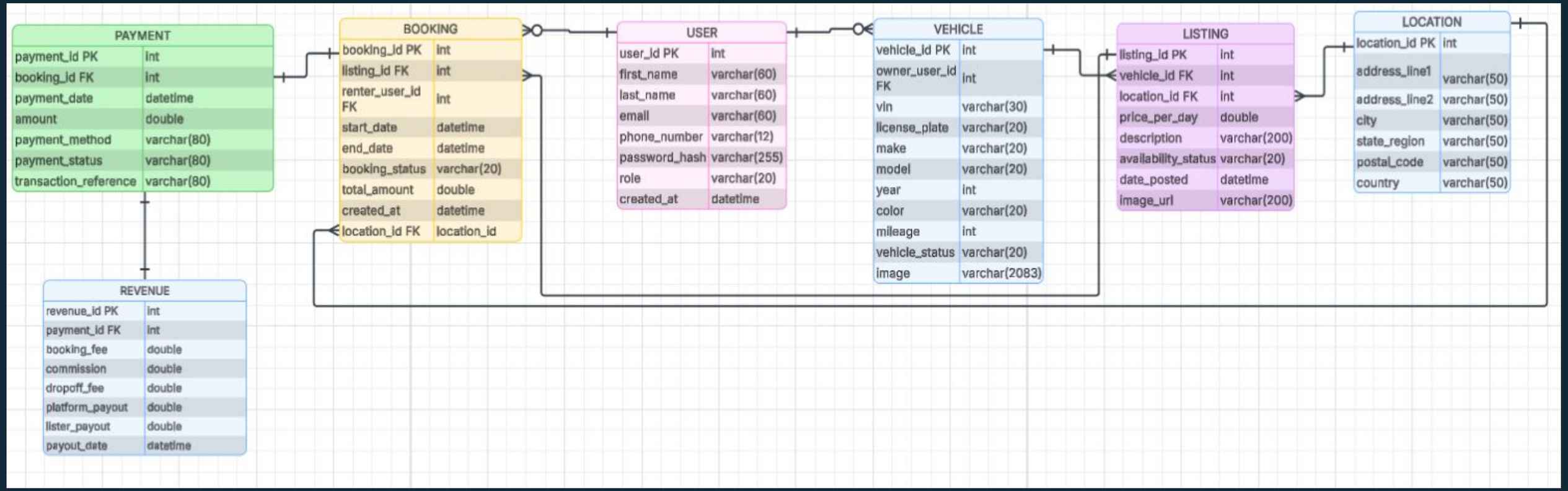
If the booking has status pending, confirmed, or completed, the vehicle becomes unavailable.

Else, if there is no booking currently on the vehicle, the vehicle's status becomes available

Trigger for Function 2: On booking_status update, or booking creation

```
103  CREATE TRIGGER trg_sync_vehicle_status_after_booking_change
104  AFTER INSERT OR UPDATE OF booking_status ON "Booking"
105  FOR EACH ROW
106  EXECUTE FUNCTION sync_vehicle_status_after_booking_change();
```

Relational Data Model

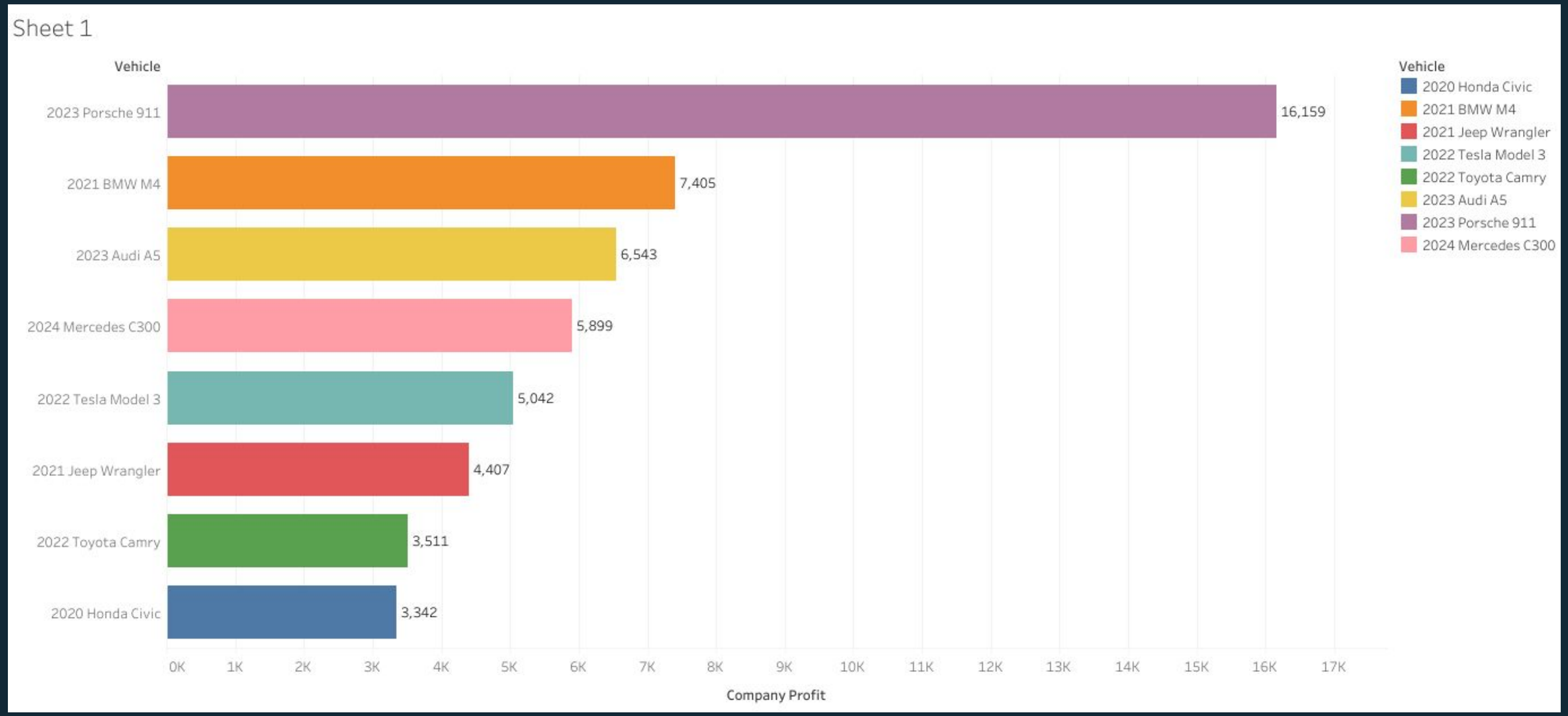


How do we make money?

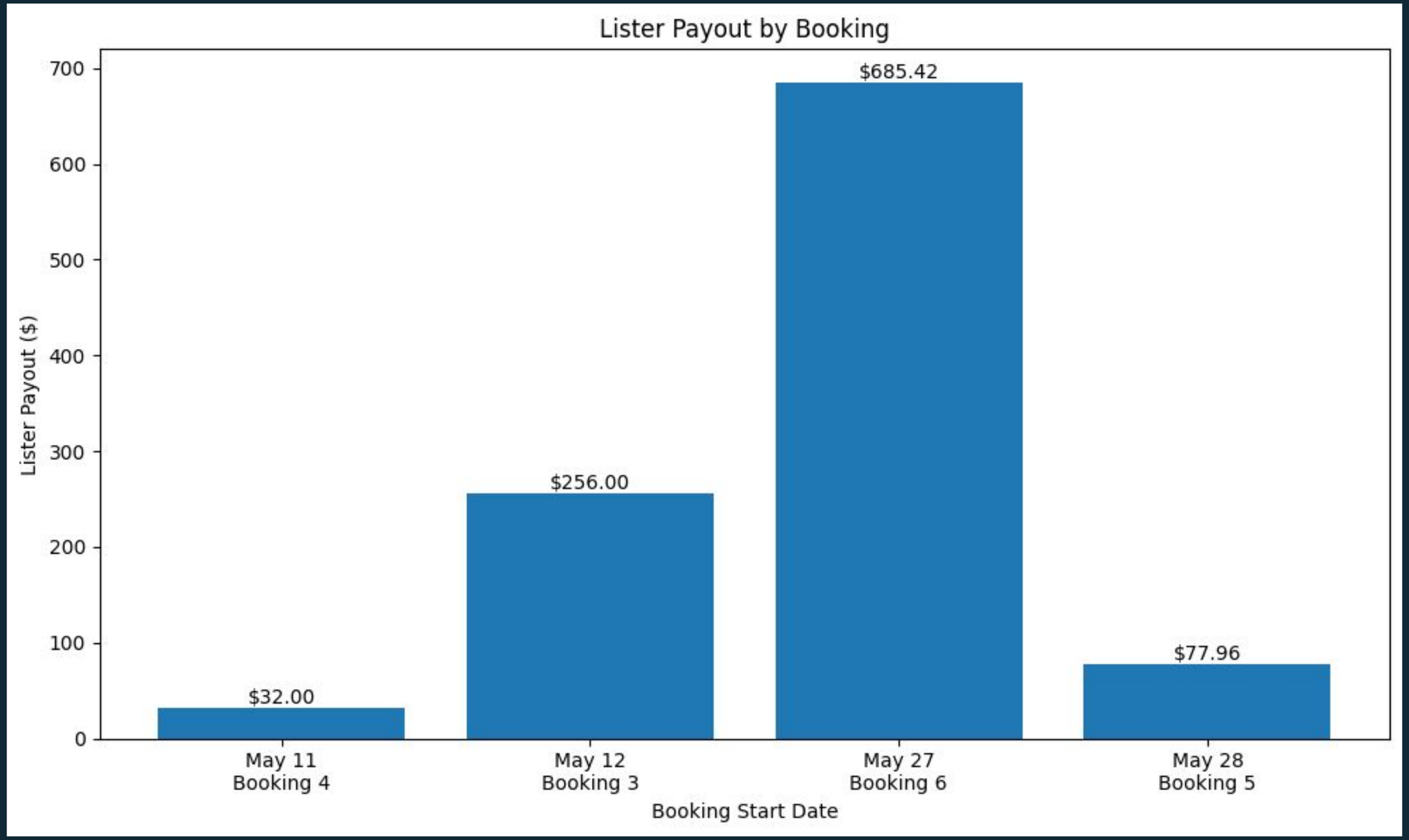
- Listings are priced per day.
- Ex: $100 per day
- Our Commission: 10%
- Delivery fee: Distance based ($1/mile)
- Booking Fee: $5 per rental



Revenue and Profits



Revenue and Profits



Extra Credit



UI/ UX

Done by Mim Dey



Tableau

Done by Rishika Datta



Docker Hosting

Done by Tyler

Takeaways

Describe a new technology you used this semester and explain why you found it especially useful or not useful at all

- Learning Docker turned out to be very useful. It allows us to package the application using a recipe script which let's users run the entire application with a single command, rather than requiring an entire setup process.

Questions?

